

TECHNICAL SPECIFICATION DOCUMENT

Digital Insurance Platform

End-to-End Online Policy Registration & Issuance System

Document Title	Technical Specification Document
Version	1.2
Revision Note	Adds customer self-service portal (policies, claims, inquiries) and ID-card upload at registration
Document Type	Technical Specification
Target Role	Fullstack Developer
Frontend	Next.js (React + TypeScript, Tailwind CSS)
Backend	Rust (Axum / Actix Web)
Database	PostgreSQL
Deployment	Docker Compose
Estimated Effort	3 – 5 Working Days

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions & Acronyms	3
1.4 Technology Stack	4
2. System Overview	5
System Components	5
3. System Architecture	6
3.1 Architecture Layers	6
3.2 End-to-End Processing Pipeline	6
4. User Roles & Permissions	7
5. Functional Workflow	8
5.1 Purchase & Issuance Journey	8
5.2 Post-Issuance Self-Service Journey	8
6. Functional Specifications	9
FS-01 · Landing Page	9
FS-02 · Customer Registration	9
FS-03 · Registration Number Generation	10
FS-04 · Invoice Generation	11
FS-05 · Email Notifications	11
FS-06 · Payment Processing (Webhook)	12
FS-07 · Policy Issuance	12
FS-08 · e-Policy PDF	13
FS-09 · Admin Authentication	13
FS-10 · Dashboard	13
FS-11 · Registration Management	14
FS-12 · Invoice Management	14
FS-13 · Policy Management	14
FS-14 · Email Log	14
FS-15 · Audit Trail	14
FS-16 · Customer Portal Authentication	14
FS-17 · Customer Portal Dashboard	15
FS-18 · Policy Viewing	15
FS-19 · Claims Submission	15
FS-20 · Policy Inquiries	16
7. Data Model & Database Design	17
customers	17
registrations	17

invoices	18
policies	18
claims	18
claim_documents	19
inquiries	19
email_logs	19
audit_logs	20
admin_users	20
8. API Specification	21
8.1 Public Endpoints	21
8.2 Customer Portal Endpoints (Customer JWT required)	21
8.3 Admin Endpoints (Admin JWT required)	21
8.4 Sample: Create Registration	22
8.5 Sample: Submit Claim	23
9. Identifier Format Specifications	24
10. Status State Machines	25
10.1 Invoice	25
10.2 Registration	25
10.3 Policy	25
10.4 Claim	25
10.5 Inquiry	25
11. Non-Functional Requirements	26
Security	26
Performance	26
Availability	26
Logging & Observability	26
12. Deployment & Environment	27
13. Deliverables	28
Source Code	28
Database	28
Documentation	28
14. Evaluation Criteria	29
15. Appendix · Bonus Features	30

1. Introduction

1.1 Purpose

This Technical Specification Document defines the architecture, data model, interfaces, and implementation requirements for the Digital Insurance Platform. It translates the product requirements into concrete engineering specifications that a fullstack developer can implement, test, and deploy without further clarification.

1.2 Scope

The platform enables prospective customers to purchase a simple, auto-accept insurance product entirely online, with no manual underwriting. Once a premium payment is confirmed, the policy is issued automatically. The system covers the following functional areas:

- Public marketing landing page
- Online customer registration
- Premium invoicing
- Transactional email notifications
- Payment confirmation via gateway webhook
- Automatic e-policy (PDF) issuance and delivery
- Customer self-service portal (login, policy viewing, claims submission, policy inquiries)
- Administrative back office (dashboard, management screens, audit trail)

1.3 Definitions & Acronyms

Term	Definition
Auto-Accept Product	An insurance product issued automatically upon payment, requiring no manual underwriting.
e-Policy	The electronic policy document delivered to the customer as a PDF.
Webhook	An HTTP callback sent by the payment gateway to notify the platform of a payment event.
JWT	JSON Web Token used to authenticate and authorize admin and customer-portal API requests.
Sum Assured	The benefit amount payable under the policy.
Coverage Term	The duration of the policy expressed in years.
Customer Portal	The authenticated self-service area where a customer views policies, submits claims, and raises inquiries.
Claim	A customer request for benefit payment against an active policy.
Inquiry	A customer question or support request related to a policy.

1.4 Technology Stack

Layer	Technology	Notes
Frontend	Next.js, React, TypeScript	SSR for the public landing page; authenticated SPAs for the customer portal and admin portal.
Styling	Tailwind CSS	Utility-first styling and responsive layout.
Backend	Rust (Axum or Actix Web)	Stateless REST API, async runtime (Tokio).
Database	PostgreSQL	Primary relational store with migrations and seed data.
PDF	Server-side rendering library	Generates the e-policy document.
Containerization	Docker Compose	Single-command local orchestration of all services.

2. System Overview

The platform is composed of loosely coupled components that communicate over HTTP. The frontend serves the public-facing experience, an authenticated customer self-service portal, and the authenticated admin portal. The backend exposes a REST API, persists data in PostgreSQL, generates PDF policies, and integrates with external email and payment-gateway services.

System Components

Component	Responsibility
Landing Page	Public marketing site introducing products and capturing intent.
Registration Service	Validates and persists customer and policy registration data.
Invoicing Service	Calculates premium, generates invoices, and tracks payment status.
Email Service	Sends transactional emails and records delivery logs.
Payment Webhook Handler	Receives and processes gateway callbacks idempotently.
Policy Service	Issues policies, generates e-policy PDFs, and manages policy status.
Customer Portal	Authenticated self-service area for viewing policies, submitting claims, and raising inquiries.
Claims Service	Records and tracks customer claims through their lifecycle.
Inquiry Service	Captures and resolves customer policy inquiries.
Admin Portal	Authenticated back office for monitoring and management.
Audit Service	Records significant system and user activities.

3. System Architecture

3.1 Architecture Layers

Layer	Description
Presentation	Next.js application (public site + admin SPA) consuming the REST API.
API / Application	Rust service exposing REST endpoints, enforcing business rules, and orchestrating workflows.
Domain / Business	Premium calculation, identifier generation, status state machines, policy issuance.
Persistence	PostgreSQL accessed via a typed query layer with migrations.
Integration	External email provider and payment gateway.

3.2 End-to-End Processing Pipeline

The core issuance pipeline is event-driven. Each step transitions the relevant entity to a new status and may emit a side effect (email, PDF generation):

Registration submitted

- > create Customer + Registration (PENDING)
- > generate Invoice (UNPAID)
- > send Registration + Invoice emails

Payment webhook received (PAID)

- > Invoice -> PAID, Registration -> PAID
- > issue Policy (ACTIVE) + render e-Policy PDF
- > send Payment Success + e-Policy emails

All state-changing operations are wrapped in database transactions, and the payment webhook is idempotent so that duplicate gateway callbacks do not issue duplicate policies.

4. User Roles & Permissions

The system recognizes two actors, each with its own authentication realm. Public actions require no authentication. After a policy is issued, the customer can authenticate to a self-service portal; the admin authenticates to the back office. Customer and admin tokens are scoped separately so that customer credentials can never access admin endpoints.

Capability	Customer	Admin
View landing page	Yes	Yes
Submit registration	Yes	No
Receive transactional emails	Yes	No
Make payment	Yes	No
Authenticate (login)	Yes	Yes
Access self-service portal	Yes	No
View own policies	Yes	Yes (all)
Download own e-Policy PDF	Yes	Yes (all)
Submit a claim	Yes	No
Track own claim status	Yes	Yes (all)
Submit a policy inquiry	Yes	No
Review / decide claims & inquiries	No	Yes
View dashboard metrics	No	Yes
View all registrations / invoices / policies	No	Yes
View email & audit logs	No	Yes

5. Functional Workflow

5.1 Purchase & Issuance Journey

The complete customer journey, expressed as a deterministic sequence of system actions:

1. Customer opens the landing page and selects a product.
2. Customer completes and submits the registration form.
3. System validates input and creates the customer and registration records.
4. System generates a unique registration number.
5. System calculates the premium and generates an invoice.
6. System sends the registration confirmation email.
7. System sends the invoice notification email.
8. Customer pays the premium through the payment gateway.
9. Payment gateway sends a webhook callback to the platform.
10. System verifies the callback and marks the invoice as PAID.
11. System updates the registration status to PAID.
12. System issues the policy and renders the e-policy PDF.
13. System sends the payment success and e-policy delivery emails.
14. Policy becomes ACTIVE; the customer's portal account is activated.

5.2 Post-Issuance Self-Service Journey

Once the policy is active, the customer can use the self-service portal:

1. Customer logs in to the portal with their email and password.
2. Customer views the list of their active and historical policies.
3. Customer opens a policy to view its details and download the e-policy PDF.
4. Customer submits a claim against an active policy, optionally attaching documents.
5. System acknowledges the claim and notifies the customer by email.
6. Admin reviews the claim and updates its status (under review, approved, rejected, paid).
7. Customer raises a policy inquiry; admin responds and the customer is notified.

6. Functional Specifications

FS-01 · Landing Page

A publicly accessible, server-rendered marketing page introducing the insurance products and driving customers to the registration flow.

Required Sections

Section	Content
Hero	Company name, tagline, and a primary call-to-action button.
Products	Life Insurance, Personal Accident Insurance, Health Insurance.
Benefits	Summary of product benefits and value proposition.
Contact	Email, phone, and physical address.
Footer	Copyright, Privacy Policy, and Terms & Conditions links.

FS-02 · Customer Registration

Customers register online through a single form composed of three field groups. The backend re-validates every field server-side regardless of client-side validation.

Personal Information

Field	Type	Required	Validation / Notes
NIK	string	Yes	Exactly 16 digits; unique across customers.
Full Name	string	Yes	Non-empty.
Birth Place	string	Yes	Non-empty.
Birth Date	date	Yes	Must not be a future date.
Gender	enum	Yes	MALE FEMALE.
Address	string	Yes	Street address.
RT/RW	string	Yes	Neighborhood unit.
Village	string	Yes	Kelurahan / Desa.
District	string	Yes	Kecamatan.
City	string	Yes	Kota / Kabupaten.
Province	string	Yes	Provinsi.
Postal Code	string	Yes	Numeric postal code.

Contact Information

Field	Type	Required	Validation / Notes
Email	string	Yes	Valid email format; unique across customers.
Mobile Number	string	Yes	Between 10 and 15 digits.

Insurance Information

Field	Type	Required	Validation / Notes
Product	enum	Yes	One of the offered products.
Sum Assured	decimal	Yes	Positive amount.
Coverage Term	integer	Yes	Coverage duration in years.

Identity Document

The registration form requires the customer to upload a scan or photo of their national ID card (KTP). The file is uploaded as part of the registration submission and stored against the customer record for verification.

Field	Type	Required	Validation / Notes
ID Card File	file	Yes	JPG, PNG, or PDF; maximum 5 MB. A single image of the front of the KTP.

The uploaded file is validated for MIME type and size, stored in a non-public location, and referenced by path on the customer record. The NIK entered in the form is the authoritative identifier; the uploaded image serves as supporting evidence.

Validation Rules Summary

Field	Rule
NIK	16 digits, unique.
Email	Valid format, unique.
Mobile Number	Minimum 10, maximum 15 digits.
Birth Date	Cannot be in the future.
ID Card File	Required; JPG/PNG/PDF; ≤ 5 MB; MIME type verified server-side.

FS-03 · Registration Number Generation

Upon successful registration the system generates a sequential, human-readable identifier scoped to the current month.

- Format: **REG-YYYYMM-NNNNNN**
- Example: **REG-202606-000001**

- NNNNNN is a zero-padded sequence that resets monthly. Generation must be concurrency-safe (e.g., a transactional counter or sequence) to avoid duplicate numbers.

FS-04 · Invoice Generation

Immediately after registration, the system creates an invoice, calculates the premium, and sets a due date.

- Invoice number format: **INV-YYYYMM-NNNNNN**
- Premium is derived from product, sum assured, and coverage term using the configured pricing rule.
- Due date is set relative to issuance (e.g., issuance date plus the configured payment window).

Invoice Fields

Field	Description
Invoice Number	Unique invoice identifier.
Registration Number	Linked registration reference.
Premium Amount	Calculated premium payable.
Due Date	Payment deadline.
Status	Current invoice state.

Invoice Status

Status	Meaning
UNPAID	Invoice issued, awaiting payment.
PAID	Payment received and confirmed.
EXPIRED	Due date passed without payment.
CANCELLED	Invoice voided.

FS-05 · Email Notifications

The system sends transactional emails at defined lifecycle events and records each send in the email log.

#	Email Type	Trigger	Attachment
1	Registration Success	Registration completed	—
2	Invoice Notification	Invoice generated	—
3	Payment Success	Payment received	—
4	e-Policy Delivery	Policy issued	Policy PDF

#	Email Type	Trigger	Attachment
5	Portal Account Activation	Policy issued (set-password link)	—
6	Claim Received	Claim submitted	—
7	Claim Status Update	Claim status changed	—
8	Inquiry Response	Admin answers an inquiry	—

FS-06 · Payment Processing (Webhook)

The platform does not collect card data directly. Payment is delegated to a gateway, which notifies the platform of the outcome through a webhook.

Endpoint

```
POST /api/payment/webhook
```

Request Body

```
{
  "invoice_no": "INV-202606-000001",
  "payment_status": "PAID",
  "payment_date": "2026-06-01"
}
```

Processing Rules

When `payment_status = PAID`, the handler performs the following within a single transaction:

1. Update the invoice status to PAID.
2. Update the registration status to PAID.
3. Issue the policy and render the e-policy PDF.
4. Send the payment success and e-policy delivery emails.

The handler must be idempotent: a repeat callback for an already-PAID invoice returns success without issuing a duplicate policy. Invalid or unknown invoice references return an appropriate error response.

FS-07 · Policy Issuance

On confirmed payment the system issues the policy automatically.

- Policy number format: **POL-YYYYMM-NNNNNN**

Policy Fields

- Policy Number
- Policy Holder Name

- Product
- Sum Assured
- Premium
- Effective Date
- Expiry Date

Policy Status

Status	Meaning
ACTIVE	Policy in force.
LAPSED	Premium obligations not maintained.
EXPIRED	Coverage term has ended.

FS-08 · e-Policy PDF

The system renders a PDF policy document containing the following sections.

Section	Fields
Policy Information	Policy number, registration number, effective date, expiry date.
Customer Information	NIK, name, birth date, address.
Coverage Information	Product, sum assured, premium.

FS-09 · Admin Authentication

Admin endpoints require authentication. The admin signs in with a username and password and receives a JWT used as a bearer token on subsequent requests.

Aspect	Specification
Credentials	Username and password.
Token	JWT (HMAC-signed) with an expiry claim.
Password Storage	Hashed with a strong adaptive algorithm (e.g., Argon2 or bcrypt).
Authorization	Bearer token required on all admin endpoints.

FS-10 · Dashboard

The dashboard surfaces key operational metrics aggregated from the data store.

- Total Registrations
- Total Invoices
- Total Paid Invoices
- Total Unpaid Invoices
- Total Policies

- Total Premium Collected

FS-11 • Registration Management

Features: list, search, filter, and pagination.

Columns: Registration No, Customer Name, Product, Status, Registration Date.

FS-12 • Invoice Management

Features: list, search, filter, and view detail.

Columns: Invoice No, Customer Name, Premium, Status, Due Date.

FS-13 • Policy Management

Features: list, search, view detail, and download PDF.

Columns: Policy No, Customer Name, Product, Status.

FS-14 • Email Log

Features: view email history with delivery status, recipient, and sent date.

Each record captures recipient, email type, delivery status (SENT / FAILED / QUEUED), and timestamp.

FS-15 • Audit Trail

All significant activities are recorded for traceability.

- Admin login
- Customer login
- Registration created
- Invoice generated
- Payment received
- Policy issued
- Claim submitted
- Claim status changed
- Inquiry submitted
- Inquiry answered
- Email sent

FS-16 • Customer Portal Authentication

After a policy is issued, the customer receives a portal-activation email containing a one-time link to set a password. Once set, the customer signs in with their registered email and password and receives a customer-scoped JWT.

Aspect	Specification
Identifier	Registered customer email address.
Activation	One-time, time-limited set-password link sent on policy issuance.
Token	Customer-scoped JWT (role = customer) with an expiry claim.
Password Storage	Hashed with a strong adaptive algorithm (Argon2 or bcrypt).
Isolation	Customer tokens cannot access admin endpoints; each customer can only access their own data.
Recovery	Password reset via a time-limited email link.

FS-17 · Customer Portal Dashboard

The portal landing view summarizes the authenticated customer's account.

- Active policy count and total sum assured
- Open claims and their current status
- Open inquiries
- Quick links to view policies, submit a claim, or raise an inquiry

FS-18 · Policy Viewing

Customers can browse and inspect only the policies linked to their own account.

Features

- List own policies with status
- View full policy detail
- Download the e-policy PDF

Columns

Policy No, Product, Sum Assured, Status, Effective Date, Expiry Date.

FS-19 · Claims Submission

Customers can file a claim against any of their active policies and track it to resolution.

Claim Form

Field	Type	Required	Notes
Policy	reference	Yes	Must be an ACTIVE policy owned by the customer.
Claim Type	enum	Yes	Depends on the product.
Incident Date	date	Yes	Not in the future; within the coverage period.
Claimed Amount	decimal	Yes	Positive; must not exceed the sum assured.

Field	Type	Required	Notes
Description	text	Yes	Free-text description of the incident.
Supporting Documents	file[]	No	Optional uploaded evidence (PDF/image).

Behavior

- A unique claim number is generated on submission (CLM-YYYYMM-NNNNNN).
- The claim starts in SUBMITTED status and a Claim Received email is sent.
- The customer can track the claim status from the portal.
- Status transitions are performed by an admin and recorded in the audit trail.

Claim Status

Status	Meaning
SUBMITTED	Claim filed, awaiting review.
UNDER_REVIEW	Admin is assessing the claim.
APPROVED	Claim accepted for payment.
REJECTED	Claim declined with a reason.
PAID	Benefit paid to the customer.

FS-20 · Policy Inquiries

Customers can raise questions about their policies and receive responses from the admin team.

Inquiry Form

Field	Type	Required	Notes
Policy	reference	No	Optional; an inquiry may be general or policy-specific.
Subject	string	Yes	Short summary of the question.
Message	text	Yes	Detailed inquiry content.

Behavior

- The inquiry starts in OPEN status.
- An admin responds; the status moves to ANSWERED and an Inquiry Response email is sent.
- Either party may close the inquiry (CLOSED).
- The customer sees the full inquiry thread in the portal.

7. Data Model & Database Design

The relational schema below captures the core entities. All tables use a UUID primary key and timestamp columns. Foreign keys enforce referential integrity between registrations, invoices, policies, claims, and inquiries. Customer-portal credentials are stored on the customers table; claims and inquiries reference the owning customer and (where applicable) policy.

customers

Column	Type	Constraints
id	UUID	PK
nik	VARCHAR(16)	UNIQUE, NOT NULL
full_name	VARCHAR	NOT NULL
birth_place	VARCHAR	NOT NULL
birth_date	DATE	NOT NULL, <= today
gender	VARCHAR	NOT NULL
address, rt_rw, village, district, city, province, postal_code	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
mobile_number	VARCHAR	NOT NULL
id_card_path	VARCHAR	NOT NULL (uploaded KTP file location)
password_hash	VARCHAR	NULLABLE (set on portal activation)
portal_status	VARCHAR	PENDING ACTIVE
created_at, updated_at	TIMESTAMPTZ	DEFAULT now()

registrations

Column	Type	Constraints
id	UUID	PK
registration_no	VARCHAR	UNIQUE, NOT NULL
customer_id	UUID	FK -> customers.id
product	VARCHAR	NOT NULL
sum_assured	NUMERIC	NOT NULL
coverage_term	INTEGER	NOT NULL (years)
status	VARCHAR	PENDING PAID ISSUED CANCELLED
created_at	TIMESTAMPTZ	DEFAULT now()

invoices

Column	Type	Constraints
id	UUID	PK
invoice_no	VARCHAR	UNIQUE, NOT NULL
registration_id	UUID	FK -> registrations.id
premium_amount	NUMERIC	NOT NULL
due_date	DATE	NOT NULL
status	VARCHAR	UNPAID PAID EXPIRED CANCELLED
paid_at	TIMESTAMPTZ	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()

policies

Column	Type	Constraints
id	UUID	PK
policy_no	VARCHAR	UNIQUE, NOT NULL
registration_id	UUID	FK -> registrations.id
product	VARCHAR	NOT NULL
sum_assured	NUMERIC	NOT NULL
premium	NUMERIC	NOT NULL
effective_date, expiry_date	DATE	NOT NULL
status	VARCHAR	ACTIVE LAPSED EXPIRED
pdf_path	VARCHAR	Generated document location
created_at	TIMESTAMPTZ	DEFAULT now()

claims

Column	Type	Constraints
id	UUID	PK
claim_no	VARCHAR	UNIQUE, NOT NULL
policy_id	UUID	FK -> policies.id
customer_id	UUID	FK -> customers.id
claim_type	VARCHAR	NOT NULL
incident_date	DATE	NOT NULL

Column	Type	Constraints
claimed_amount	NUMERIC	NOT NULL
description	TEXT	NOT NULL
status	VARCHAR	SUBMITTED UNDER_REVIEW APPROVED REJECTED PAID
decision_note	TEXT	NULLABLE
submitted_at, updated_at	TIMESTAMPTZ	DEFAULT now()

claim_documents

Column	Type	Constraints
id	UUID	PK
claim_id	UUID	FK -> claims.id
file_name	VARCHAR	NOT NULL
file_path	VARCHAR	NOT NULL
uploaded_at	TIMESTAMPTZ	DEFAULT now()

inquiries

Column	Type	Constraints
id	UUID	PK
inquiry_no	VARCHAR	UNIQUE, NOT NULL
customer_id	UUID	FK -> customers.id
policy_id	UUID	FK -> policies.id, NULLABLE
subject	VARCHAR	NOT NULL
message	TEXT	NOT NULL
status	VARCHAR	OPEN ANSWERED CLOSED
response	TEXT	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()
responded_at	TIMESTAMPTZ	NULLABLE

email_logs

Column	Type	Constraints
id	UUID	PK
recipient	VARCHAR	NOT NULL

Column	Type	Constraints
email_type	VARCHAR	Registration Invoice Payment Policy Activation Claim Inquiry
subject	VARCHAR	NOT NULL
status	VARCHAR	SENT FAILED QUEUED
error_message	TEXT	NULLABLE
sent_at	TIMESTAMPTZ	NULLABLE

audit_logs

Column	Type	Constraints
id	UUID	PK
actor	VARCHAR	User or system identifier
action	VARCHAR	NOT NULL
entity_type	VARCHAR	Registration Invoice Policy Claim Inquiry ...
entity_id	UUID	NULLABLE
metadata	JSONB	Contextual payload
ip_address	VARCHAR	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()

admin_users

Column	Type	Constraints
id	UUID	PK
username	VARCHAR	UNIQUE, NOT NULL
password_hash	VARCHAR	NOT NULL
role	VARCHAR	DEFAULT 'admin'
created_at	TIMESTAMPTZ	DEFAULT now()

8. API Specification

The backend exposes a REST API. Public endpoints are unauthenticated; customer-portal and admin endpoints each require a valid, role-scoped JWT bearer token. All responses are JSON; errors use appropriate HTTP status codes with a consistent error envelope.

8.1 Public Endpoints

Method	Endpoint	Description
GET	/api/products	List available insurance products.
POST	/api/registrations	Submit a new customer registration (multipart, includes ID card file).
GET	/api/registrations/{regNo}	Retrieve registration status by number.
POST	/api/payment/webhook	Receive a payment gateway callback.

8.2 Customer Portal Endpoints (Customer JWT required)

Method	Endpoint	Description
POST	/api/customer/activate	Set initial password using the activation token.
POST	/api/customer/login	Authenticate and obtain a customer JWT.
POST	/api/customer/password/reset	Request or complete a password reset.
GET	/api/customer/me	Current customer profile & portal summary.
GET	/api/customer/policies	List the customer's own policies.
GET	/api/customer/policies/{id}	Policy detail (owned by the customer).
GET	/api/customer/policies/{id}/pdf	Download the customer's e-policy PDF.
POST	/api/customer/claims	Submit a claim against an active policy.
GET	/api/customer/claims	List the customer's claims.
GET	/api/customer/claims/{id}	Claim detail & status.
POST	/api/customer/inquiries	Submit a policy inquiry.
GET	/api/customer/inquiries	List the customer's inquiries.
GET	/api/customer/inquiries/{id}	Inquiry detail & response thread.

8.3 Admin Endpoints (Admin JWT required)

Method	Endpoint	Description
POST	/api/admin/login	Authenticate and obtain an admin JWT.

Method	Endpoint	Description
GET	/api/dashboard/stats	Aggregated dashboard metrics.
GET	/api/registrations	List registrations (search, filter, paginate).
GET	/api/registrations/{id}	Registration detail.
GET	/api/invoices	List invoices.
GET	/api/invoices/{id}	Invoice detail.
GET	/api/policies	List all policies.
GET	/api/policies/{id}	Policy detail.
GET	/api/policies/{id}/pdf	Download any e-policy PDF.
GET	/api/claims	List all claims (search, filter).
PATCH	/api/claims/{id}	Update claim status & decision note.
GET	/api/inquiries	List all inquiries.
POST	/api/inquiries/{id}/respond	Respond to a customer inquiry.
GET	/api/email-logs	List email delivery logs.
GET	/api/audit-logs	List audit trail entries.

8.4 Sample: Create Registration

The registration is submitted as multipart/form-data so the ID card file can be uploaded alongside the form fields.

```
POST /api/registrations
Content-Type: multipart/form-data; boundary=----X

-----X
Content-Disposition: form-data; name="data"
Content-Type: application/json

{
  "nik": "3201010101010001",
  "full_name": "Budi Santoso",
  "email": "budi@example.com",
  "mobile_number": "081234567890",
  "product": "LIFE",
  "sum_assured": 100000000,
  "coverage_term": 10
}
-----X
Content-Disposition: form-data; name="id_card";
  filename="ktp.jpg"
Content-Type: image/jpeg
```

```
<binary image data>
```

```
-----X--
```

```
201 Created
```

```
{  
  "registration_no": "REG-202606-000001",  
  "invoice_no": "INV-202606-000001",  
  "status": "PENDING"  
}
```

8.5 Sample: Submit Claim

```
POST /api/customer/claims
```

```
Authorization: Bearer <customer_jwt>
```

```
Content-Type: application/json
```

```
{  
  "policy_id": "9f1c...",  
  "claim_type": "ACCIDENT",  
  "incident_date": "2026-05-20",  
  "claimed_amount": 5000000,  
  "description": "Hospitalization following an accident."  
}
```

```
201 Created
```

```
{  
  "claim_no": "CLM-202606-000001",  
  "status": "SUBMITTED"  
}
```


9. Identifier Format Specifications

Entity	Format	Example
Registration	REG-YYYYMM-NNNNNN	REG-202606-000001
Invoice	INV-YYYYMM-NNNNNN	INV-202606-000001
Policy	POL-YYYYMM-NNNNNN	POL-202606-000001
Claim	CLM-YYYYMM-NNNNNN	CLM-202606-000001
Inquiry	INQ-YYYYMM-NNNNNN	INQ-202606-000001

YYYYMM is the issuance year and month. NNNNNN is a six-digit, zero-padded sequence that resets at the start of each month. Sequence allocation must be transactional and concurrency-safe.

10. Status State Machines

10.1 Invoice

UNPAID --payment confirmed--> PAID
UNPAID --due date passed--> EXPIRED
UNPAID --voided--> CANCELLED

10.2 Registration

PENDING --invoice paid--> PAID --policy issued--> ISSUED
PENDING --voided--> CANCELLED

10.3 Policy

ACTIVE --premium not maintained--> LAPSED
ACTIVE --term ends--> EXPIRED

10.4 Claim

SUBMITTED --admin opens--> UNDER_REVIEW
UNDER_REVIEW --accepted--> APPROVED --benefit paid--> PAID
SUBMITTED | UNDER_REVIEW --declined--> REJECTED

10.5 Inquiry

OPEN --admin responds--> ANSWERED --resolved--> CLOSED

11. Non-Functional Requirements

Security

- Role-scoped JWT authentication for customer-portal and admin endpoints.
- Ownership enforcement so customers can access only their own policies, claims, and inquiries.
- Passwords hashed with a strong adaptive algorithm.
- Time-limited, single-use tokens for portal activation and password reset.
- Uploaded identity documents (ID card) and claim files stored in non-public storage with validated MIME type and size limits, served only via authorized endpoints.
- Server-side input validation on all endpoints.
- Parameterized queries to prevent SQL injection.
- HTTPS enforced in production environments.

Performance

- Target API response time: < **2 seconds** under nominal load.

Availability

- The full stack must run locally with a single **docker compose up**.

Logging & Observability

- Application logs for request/response lifecycle.
- Error logs with stack context.
- Audit logs for business-significant events.

12. Deployment & Environment

The application is orchestrated with Docker Compose, defining the frontend, backend, and PostgreSQL services. Configuration is supplied through environment variables.

Variable	Purpose
DATABASE_URL	PostgreSQL connection string.
JWT_SECRET	Signing secret for admin tokens.
SMTP_* / EMAIL_API_KEY	Email provider credentials.
PAYMENT_WEBHOOK_SECRET	Shared secret to verify gateway callbacks.
APP_BASE_URL	Public base URL for links in emails.

13. Deliverables

Source Code

- Frontend repository (Next.js).
- Backend repository (Rust).

Database

- Migration scripts.
- Seed data.

Documentation

A README.md covering:

- Application setup.
- Database setup.
- Running the application.
- API documentation.

14. Evaluation Criteria

Criteria	Weight
Frontend Development	25%
Backend Development	25%
Database Design	15%
API Design	15%
Code Quality	10%
Security	5%
Documentation	5%

Estimated Effort: 3 – 5 Working Days

15. Appendix · Bonus Features

Additional credit is awarded for implementing any of the following:

- Docker Compose orchestration
- Swagger / OpenAPI documentation
- Unit testing
- Integration testing
- Background job processing
- Email queue
- Dashboard charts
- Role-based access control (RBAC)
- CI/CD pipeline